

Topic-1: Operators in Java

Java provides a rich set of operators to manipulate variables. We can divide all the Java operators into the following groups –

- Arithmetic Operators
- Relational Operators
- Bitwise Operators
- Logical Operators
- Assignment Operators
- Misc Operators

1. Arithmetic Operators

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The following table lists the arithmetic operators –

Assume integer variable A holds 10 and variable B holds 20, then

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator.	A + B will give 30
- (Subtraction)	Subtracts right-hand operand from left-hand operand.	A - B will give -10
* (Multiplication)	Multiplies values on either side of the operator.	A * B will give 200
/ (Division)	Divides left-hand operand by right-hand operand.	B / A will give 2
% (Modulus)	Divides left-hand operand by right-hand operand and returns remainder.	B % A will give 0
++ (Increment)	Increases the value of operand by 1.	B++ gives 21
-- (Decrement)	Decreases the value of operand by 1.	B-- gives 19

2. Relational Operators

There are following relational operators supported by Java language.

Assume variable A holds 10 and variable B holds 20, then –

Operator	Description	Example
== (equal to)	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!= (not equal to)	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
> (greater than)	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
< (less than)	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>= (greater than or equal to)	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<= (less than or equal to)	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

3. Bitwise Operators

Java defines several bitwise operators, which can be applied to the integer types, long, int, short, char, and byte.

Bitwise operator works on bits and performs bit-by-bit operation.

Assume if a = 60 and b = 13; now in binary format they will be as follows –

a = 0011 1100

b = 0000 1101

a&b = 0000 1100

a|b = 0011 1101

a^b = 0011 0001

$\sim a = 1100\ 0011$

The following table lists the bitwise operators –

Assume integer variable A holds 60 and variable B holds 13 then

Operator	Description	Example
& (bitwise and)	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
(bitwise or)	Binary OR Operator copies a bit if it exists in either operand.	(A B) will give 61 which is 0011 1101
^ (bitwise XOR)	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49 which is 0011 0001
~ (bitwise compliment)	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<< (left shift)	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>> (right shift)	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 1111
>>> (zero fill right shift)	Shift right zero fill operator. The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.	A >>>2 will give 15 which is 0000 1111

4. Logical Operators

The following table lists the logical operators –

Assume Boolean variables A holds true and variable B holds false, then

Operator	Description	Example
&& (logical and)	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false
(logical or)	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A B) is true
! (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true

5. Assignment Operators

Following are the assignment operators supported by Java language

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand.	C = A + B will assign value of A + B into C
+=	Add AND assignment operator. It adds right operand to the left operand and assign the result to left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts right operand from the left operand and assign the result to left operand.	C -= A is equivalent to C = C – A
*=	Multiply AND assignment operator. It multiplies right operand with the left operand and assign the result to left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator. It divides left operand with the right operand and assign the result to left operand.	C /= A is equivalent to C = C / A

<code>%=</code>	Modulus AND assignment operator. It takes modulus using two operands and assign the result to left operand.	<code>C %= A</code> is equivalent to <code>C = C % A</code>
<code><<=</code>	Left shift AND assignment operator.	<code>C <<= 2</code> is same as <code>C = C << 2</code>
<code>>>=</code>	Bitwise AND assignment operator.	<code>C >>= 2</code> is same as <code>C = C >> 2</code>
<code>&=</code>	Right shift AND assignment operator.	<code>C &= 2</code> is same as <code>C = C & 2</code>
<code>^=</code>	bitwise exclusive OR and assignment operator.	<code>C ^= 2</code> is same as <code>C = C ^ 2</code>
<code> =</code>	bitwise inclusive OR and assignment operator.	<code>C = 2</code> is same as <code>C = C 2</code>

6. Miscellaneous Operators

There are few other operators supported by Java Language.

a. Conditional Operator (? :)

- Conditional operator is also known as the **ternary operator**.
- This operator consists of three operands and is used to evaluate Boolean expressions.
- The goal of the operator is to decide, which value should be assigned to the variable.

Syntax:

variable x = (expression) ? value if true : value if false

Following is an example –

Example

```
public class Test {
    public static void main(String args[]) {
        int a, b;
        a = 10;
        b = (a == 1) ? 20: 30;
        System.out.println( "Value of b is : " + b );
        b = (a == 10) ? 20: 30;
        System.out.println( "Value of b is : " + b );
    }
}
```

Output

Value of b is : 30

Value of b is : 20

b. instanceof Operator

- This operator is used only for object reference variables.
- The operator checks whether the object is of a particular type (class type or interface type).

Syntax:

(Object reference variable) instanceof (class/interface type)

If the object referred by the variable on the left side of the operator passes the IS-A check for the class/interface type on the right side, then the result will be true.

Example

```
public class Test {  
    public static void main(String args[]) {  
        String name = "James";  
        // following will return true since name is type of String  
        boolean result = name instanceof String;  
        System.out.println( result );  
    }  
}
```

Output

true

This operator will still return true, if the object being compared is the assignment compatible with the type on the right. Following is one more example –

Example

```
class Vehicle {}  
  
public class Car extends Vehicle {  
    public static void main(String args[]) {  
        Vehicle a = new Car();  
        boolean result = a instanceof Car;  
        System.out.println( result );  
    }  
}
```

Output

true

Topic-2 Precedence of Java Operators

- Operator precedence determines the grouping of terms in an expression.
- This affects how an expression is evaluated.
- Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator –

Example $x = 7 + 3 * 2$;

here x is assigned 13, not 20 because operator * has higher precedence than +, so it first gets multiplied with $3 * 2$ and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Category	Operator	Associativity
Postfix	>() [] . (dot operator)	Left to right
Unary	>++ - - ! ~	Right to left
Multiplicative	>* /	Left to right
Additive	>+ -	Left to right
Shift	>>>> >>>> <<<<	Left to right
Relational	>> >= < <=	Left to right
Equality	>== !=	Left to right
Bitwise AND	>&	Left to right
Bitwise XOR	>^	Left to right
Bitwise OR	>	Left to right
Logical AND	>&&	Left to right
Logical OR	>	Left to right
Conditional	>?:	Right to left
Assignment	>= += -= *= /= %= >>= <<= &= ^= =	Right to left

Topic-3: Java Control Statements

- A control statement works as a determiner for deciding the next task of the other statements whether to execute or not.
- If we use a control statement, we can set an order for executing a program based on values and logic.
- In Java, the control statements are divided into three categories
 - selection (or) decision making statements
 - iteration (or) loop statements
 - jump statements.

1. Decision Making statements in Java

- simple if Statement
- if...else Statement
- Nested if statement
- if...else if...else statement
- switch statement

2. Looping Statements in Java

- while loop
- do...while loop
- for loop
- for-each loop

3. Branching Statements in Java

- break
- continue

1. Decision Making Statements in Java

Decision-making statements are statements that decide what to execute and when. They are similar to decision making in real-time. Control flow statements control the flow of a program's execution. Here flow of execution will be based on the state of a program. We have 4 decision-making statements available in Java.

a) Simple if Statement

Simple if the statement is the basis of decision-making statements in Java. It decides if a certain amount of code should be executed based on the condition.

Syntax:

```
if (condition) {  
    Statement 1; //if condition becomes true then this will be executed  
}  
Statement 2; //this will be executed irrespective of condition becomes true  
or false
```

Example:

```
class Test  
{  
    public static void main(String args[])  
    {  
        int x = 5;  
        if (x > 10)  
            System.out.println("Inside If");  
        System.out.println("After if statement");  
    }  
}
```

Output:

After if statement

b) if...else Statement

In if...else statement, if the condition is true then statements in if block will be executed but if it comes out as false then else block will be executed.

Syntax:

```
if (condition) {  
    Statement 1; //if condition becomes true then this will be executed  
}
```

Example:

```
class Test  
{  
    public static void main(String args[])  
    {  
        int x = 9;  
        if (x > 10)  
            System.out.println("i is greater than 10");  
        else  
            System.out.println("i is less than 10");  
        System.out.println("After if else statement");  
    }  
}
```

Output:

i is less than 10
After if else statement

c) Nested if statement

Nested if statement is if inside an if block. It is same as normal if...else statement but they are written inside another if...else statement.

Syntax:

```
if (condition1) {  
    Statement 1; //executed when condition1 is true  
    if (condition2) {  
        Statement 2; //executed when condition2 is true  
    }  
    else {  
        Statement 3; //executed when condition2 is false  
    }  
}
```

Example:

```
class Test  
{  
    public static void main(String args[])  
    {  
        int x = 25;  
  
        if (x > 10)  
        {  
            if (x%2==0)  
                System.out.println("i is greater than 10 and even number");  
            else  
                System.out.println("i is greater than 10 and odd number");  
        }  
        else  
        {  
            System.out.println("i is less than 10");  
        }  
        System.out.println("After nested if statement");  
    }  
}
```

Output:

i is greater than 10 and odd number
After nested if statement

d) if...else if...else statement

if...else if statements will be used when we need to compare the value with more than 2 conditions. They are executed from top to bottom approach. As soon as the code finds the matching condition, that block will be executed. But if no condition is matching then the last else statement will be executed.

Syntax:

```
if (condition2) {
```

```
Statement 1; //if condition1 becomes true then this will be executed
}
else if (condition2) {
    Statement 2; // if condition2 becomes true then this will be executed
}
.
.
else {
    Statement 3; //executed when no matching condition found
}
```

Example:

```
class Test
{
    public static void main(String args[])
    {
        int x = 2;

        if (x > 10)
        {
            System.out.println("i is greater than 10");
        }
        else if (x < 10)
            System.out.println("i is less than 10");
        }
        else
        {
            System.out.println("i is 10");
        }
        System.out.println("After if else if ladder statement");
    }
}
```

Output:

```
i is less than 10
After if else if ladder statement
```

e) Switch statement

Java switch statement compares the value and executes one of the case blocks based on the condition. It is same as if...else if ladder. Below are some points to consider while working with switch statements:

- case value must be of the same type as expression used in switch statement
- case value must be a constant or literal. It doesn't allow variables
- case values should be unique. If it is duplicate, then program will give compile time error

Let us understand it through one example.

```
class SwitchDemo{
    public static void main(String args[]){
        int i=2;
        switch(i){
            case 0:
```

```
        System.out.println("i is 0");
        break;

        case 1:
        System.out.println("i is 1");
        break;

        case 2:
        System.out.println("i is 2");
        break;

        case 3:
        System.out.println("i is 3");
        break;

        case 4:
        System.out.println("i is 4");
        break;

        default:
        System.out.println("i is not in the list");
        break;
    }
}
```

2. Looping Statements in Java

Looping statements are the statements which executes a block of code repeatedly until some condition meet to the criteria. Loops can be considered as repeating if statements. There are 3 types of loops available in Java.

a) while

while loops are simplest kind of loop. It checks and evaluates the condition and if it is true then executes the body of loop. This is repeated until the condition becomes false. Condition in while loop must be given as a Boolean expression. If int or string is used instead, compile will give the error.

Syntax:

```
while (condition)
{
    statement1;
}
```

Example:

```
class Test
{
    public static void main(String args[])
    {
        int j = 1;
        while (j <= 10)
        {
```

```
        System.out.println(j);
        j = j+2;
    }
}
```

Output:

```
1
3
5
7
9
```

b) do...while

Do...while works same as while loop. It has only one difference that in do...while condition is checked after the execution of the loop body. That is why this loop is considered as exit control loop. In do...while loop, body of loop will be executed at least once before checking the condition

Syntax:

```
do{
statement1;
}while(condition);
Example: here
class Test {
    public static void main(String args[]) {
        int j = 10;

        do{
            System.out.println(j);
            j = j+1;
        } while (j <= 10)
    }
}
```

Output:

```
10
```

c) for

It is the most common and widely used loop in Java. It is the easiest way to construct a loop structure in code as initialization of a variable, a condition and increment/decrement are declared only in a single line of code. It is easy to debug structure in Java.

Syntax:

```
for (initialization; condition; increment/decrement) {
    statement;
}
```

Example:

```
class Test
{
    public static void main(String args[])
    {
        for (int j = 1; j <= 5; j++)
            System.out.println(j);
    }
}
```

Output:

```
1
2
3
4
5
```

d) for-each Loop

For-Each loop is used to traverse through elements in an array. It is easier to use because we don't have to increment the value. It returns the elements from the array or collection one by one.

Example:

```
class Demo{
    public static void main(String args[]){
        int a[] = {10,15,20,25,30};
        for (int i : a) {
            System.out.println(i);
        }
    }
}
```

Output:

```
10
15
20
25
30
```

3. Branching Statements in Java

Branching statements jump from one statement to another and transfer the execution flow. There are 2 branching statements in Java.

a) Break

Break statement is used to terminate the execution and bypass the remaining code in loop. It is mostly used in loop to stop the execution and comes out of loop. When there are nested loops then break will terminate the innermost loop.

Example:

```
class Test
{
    public static void main(String args[])
    {
        for (int j = 0; j < 5; j++)
        {
            // come out of loop when i is 4.
            if (j == 4)
                break;
            System.out.println(j);
        }
        System.out.println("After loop");
    }
}
```

Output:

```
0
1
2
3
4
After loop
```

b) Continue

Continue statement works same as break but the difference is it only comes out of loop for that iteration and continue to execute the code for next iterations. So it only bypasses the current iteration.

Example:

```
class Test
{
    public static void main(String args[])
    {
        for (int j = 0; j < 10; j++)
        {
            // If the number is odd then bypass and continue with next
            value
            if (j%2 != 0)
                continue;
            // only even numbers will be printed
            System.out.print(j + " ");
        }
    }
}
```

Output:

```
0 2 4 6 8
```